# Restful Booker Platform — Technical Assessment

Quality Assurance & SDET Engineering Evaluation Report

| CANDIDATE | TARGET APP | FRAMEWORK | STATUS |
|---|---|---|---|
| QA / SDET Engineer | automationintesting.online | Playwright BDD (OOP POM) | Completed & Verified |

## 01 Test Case Design & Matrix (Task 1)

**Selected Workflow:** `Room Booking & Reservation Workflow` (Core Business Revenue Stream).
Covers inventory availability checking, stay dates selection, guest credentials submission, and automated confirmation issuance.

| TEST ID | SCENARIO SUMMARY | TYPE | PRIORITY | PRECONDITION & STEPS | EXPECTED RESULT |
|---|---|---|---|---|---|
| TC-BOOK-001 | Successful room booking with valid guest details | POSITIVE E2E | P1 | User on reservation page with available dates → Fills valid guest info → Clicks Reserve | HTTP 201 Created. Booking confirmed card appears with dates and "Return home" button. |
| TC-BOOK-002 | Submit booking with all mandatory fields empty | NEGATIVE | P1 | User opens booking modal → Leaves all fields blank → Clicks Reserve Now | HTTP 400 Bad Request. Red alert banner lists mandatory field errors for Firstname, Lastname, Email, Phone. |
| TC-BOOK-003 | Guest name below minimum boundary (< 3 chars) | BOUNDARY | P2 | Inputs 2 chars ("Jo", "Do") → Valid email & phone → Submits | Form rejected with "size must be between 3 and 18". Booking is blocked. |
| TC-BOOK-004 | Guest name at maximum boundary (18 chars) | BOUNDARY | P2 | Inputs exactly 18 chars for names → Valid email & phone → Submits | HTTP 201 Created. Boundary limit is accepted cleanly without error. |
| TC-BOOK-005 | Invalid email format (missing '@' & domain) | NEGATIVE | P1 | Inputs valid names and phone → Inputs "invalidemail" → Submits | Form rejected with email format validation notice. Booking is blocked. |
| TC-BOOK-006 | Phone number below minimum boundary (10 digits) | BOUNDARY | P1 | Inputs 10-digit number ("0123456789") → Other fields valid → Submits | Form rejected with error "size must be between 11 and 21". |
| TC-BOOK-007 | Phone number at boundaries (11 & 21 digits) | BOUNDARY | P2 | Tests with 11-digit and 21-digit valid numeric phones → Submits | HTTP 201 Created. Both boundary lengths succeed. |
| TC-BOOK-008 | Double-booking on overlapping reserved dates | CONFLICT | P1 | Room has existing booking → Second guest books same dates → Submits | HTTP 409 Conflict. In-modal alert informs room is unavailable (See Defect BUG-001). |
| TC-BOOK-009 | Minimum stay duration (1-night reservation) | BOUNDARY | P2 | Selects 1 night stay (Checkin + 1 day) → Submits reservation | Price correctly calculates 1 night + cleaning + service fee. Booking confirmed. |
| TC-BOOK-010 | Check-out date earlier than Check-in date | EDGE CASE | P1 | Attempts reverse date selection via calendar or URL parameters | Calendar disallows selection OR form blocks with "Checkout must be after Checkin". |

| TEST ID | SCENARIO SUMMARY | TYPE | PRIORITY | PRECONDITION & STEPS | EXPECTED RESULT |
|---|---|---|---|---|---|
| **TC-BOOK-011** | Phone number containing alphabetical characters | EDGE CASE | P2 | Inputs "abcdefghijk12" in phone input → Submits form | Validation error: "Phone must contain only numeric digits" (See Defect BUG-002). |
| **TC-BOOK-012** | Guest names with international characters / accents | EDGE CASE | P3 | Inputs Firstname "René" and Lastname "O'Connor" → Submits | HTTP 201 Created. Special characters handled cleanly without corruption. |

## 01.1 Executable Gherkin BDD Specifications (Jira Xray Ready)

Formatted in pure Cucumber/Gherkin syntax matching the automated test suite in `features/booking.feature`.

```
@booking @hotel-management @regression
Feature: Room Reservation Workflow - Shady Meadows B&B
  As a prospective guest of Shady Meadows B&B
  I want to select and book a hotel room online
  So that I have a confirmed reservation for my upcoming stay

  # --- POSITIVE E2E SCENARIO ---
  @TC-BOOK-001 @XRAY-101 @smoke @positive @e2e
  Scenario: [XRAY-101] Successfully book a room with valid guest details
    Given the guest navigates to the reservation page for an available room
    When the guest opens the reservation modal
    And the guest enters valid guest details
    And the guest submits the reservation form
    Then the system displays the booking confirmation with the reserved dates

  # --- NEGATIVE VALIDATION SCENARIO ---
  @TC-BOOK-002 @XRAY-102 @negative @validation
  Scenario: [XRAY-102] Display validation errors when submitting booking form with missing mandatory fields
    Given the guest navigates to the reservation page for an available room
    When the guest opens the reservation modal
    And the guest leaves all guest details fields empty
    And the guest submits the reservation form
    Then the system displays a validation error banner containing mandatory field requirements

  # --- BOUNDARY SCENARIO ---
  @TC-BOOK-003 @negative @boundary
  Scenario Outline: System rejects guest names below minimum boundary limit (< 3 characters)
    Given User has opened the booking reservation modal
    When User enters Firstname "<firstname>" and Lastname "<lastname>"
    And User clicks the "Reserve Now" button inside the modal
    Then The form submission should be rejected with validation error "size must be between 3 and 18"

    Examples:
      | firstname | lastname | length_description |
      | Jo        | Do       | 2 characters each  |
      | A         | B        | 1 character each   |

  # --- DATE CONFLICT SCENARIO ---
  @TC-BOOK-008 @negative @conflict
  Scenario: System prevents double-booking when selected dates overlap with an existing confirmed reservation
    Given Room "101" already has a confirmed booking for dates "2026-11-10" to "2026-11-12"
    When Another guest attempts to book Room "101" for overlapping dates "2026-11-10" to "2026-11-12"
    And The guest submits valid guest information
    Then The backend API should reject the booking with HTTP 409 Conflict
    And The UI should display a graceful alert indicating the dates are unavailable
```

## 02 Structured Defect Reports (Task 1)

Documenting three (3) critical and medium findings identified during exploratory testing and test automation against the live application.

### BUG-001 Frontend React Crashes to Error Boundary ("This page couldn't load") on Date Conflict (HTTP 409)

CRITICAL (S1) HIGH (P1)

**STEPS TO REPRODUCE:**

1. Navigate to: `https://automationintesting.online/reservation/1?checkin=2026-11-10&checkout=2026-11-12` (dates with existing booking).
2. Click the blue **"Reserve Now"** button on the price card.
3. Fill valid guest details: Firstname: `John`, Lastname: `Doe`, Email: `john@example.com`, Phone: `0812345678901`.
4. Click **"Reserve Now"** inside the modal.

**EXPECTED RESULT**

Modal stays open and displays an in-modal warning banner: *"The selected dates are already booked or unavailable. Please choose alternative dates."* User input is preserved.

**ACTUAL RESULT**

API returns `HTTP 409 Conflict`. React frontend unhandled Promise rejection crashes the page into: **"This page couldn't load. Reload to try again, or go back."** All user input is lost.

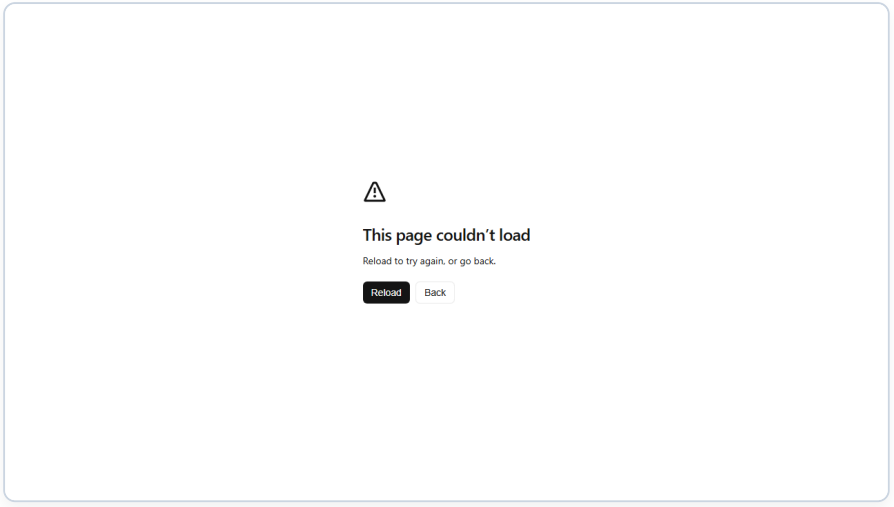



*Figure 2.1: Full page React crash boundary upon receiving HTTP 409 Conflict response.*

## BUG-002 Missing Phone Number Format Validation (Accepts Purely Alphabetic Strings)

MEDIUM (S2) | MEDIUM (P2)

**STEPS TO REPRODUCE:**

1. Send `POST /api/booking` with payload containing alphabetic phone: `"phone": "abcdefghijk12"` (13 chars).
2. Observe HTTP response code and database entry.

**EXPECTED RESULT**

API rejects submission with `HTTP 400 Bad Request` : *"Phone number must only contain valid numeric digits and dialing symbols."*

**ACTUAL RESULT**

API responds with `HTTP 201 Created` and generates a new reservation ID. Backend only checks string length (11–21 chars) without regex character validation.

## BUG-003 Raw Unmapped Backend Error Messages Displayed in UI with Duplicates and Missing Field Names

LOW / UX (S3) | MEDIUM (P2)

**STEPS TO REPRODUCE:**

1. Open reservation modal → Leave all fields blank → Click **"Reserve Now"**.
2. Observe the red validation alert box.

**EXPECTED RESULT**

Validation notices displayed cleanly under their respective fields (e.g. *"Firstname is required"*, *"Phone must be between 11–21 digits"*).

**ACTUAL RESULT**

UI dumps unformatted server validation strings with duplicate lines: `must not be empty` appears twice, and `size must be between...` does not mention field names.

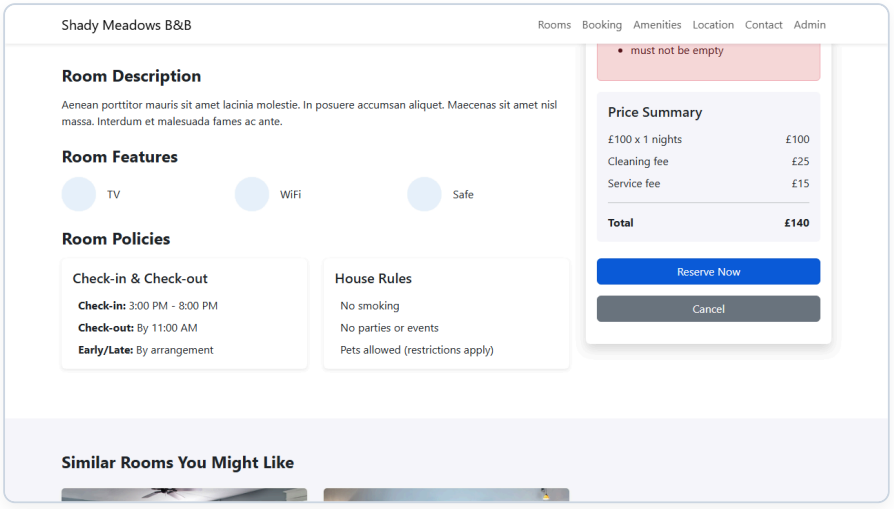



*Figure 2.2: Red error banner with duplicate error strings and ambiguous field constraints.*

## 03 API Testing Strategy & Artifacts (Task 2)

Identified microservice endpoints: Auth, Room, Booking, Report, and Message. Exported Postman collection contains 11 automated test requests with dynamic token chaining.

| SCENARIO | REQUEST NAME & ENDPOINT | METHOD | AUTH STATE | STATUS | KEY ASSERTIONS & SCRIPTS |
|---|---|---|---|---|---|
| **API-001** | Admin Login | `POST /api/auth/login` | None | 200 OK | Extracts `token`, saves to `{{authToken}}` environment variable. |
| **API-002** | Invalid Login | `POST /api/auth/login` | None | 401 / 403 | Verifies unauthorized access rejection on bad credentials. |
| **API-003** | Get All Rooms | `GET /api/room` | None | 200 OK | Asserts `rooms` array not empty, verifies roomid, roomPrice, and type. |
| **API-004** | Get Room by ID | `GET /api/room/:id` | None | 200 OK | Asserts returned room ID matches requested environment parameter. |
| **API-005** | Get Non-existent Room | `GET /api/room/99999` | None | 404 / empty | Defensive assertion verifying resource absence handling. |
| **API-006** | Create Booking (Valid) | `POST /api/booking` | None | 201 Created | Pre-request generates unique future dates. Captures `bookingid`. |
| **API-007** | Create Booking (Empty) | `POST /api/booking` | None | 400 Bad Req | Asserts `errors` array returns mandatory field validation rules. |
| **API-008** | Create Booking (Conflict) | `POST /api/booking` | None | 409 Conflict | Double booking rejection: asserts `{"error":"Failed to create booking"}`. |
| **API-009** | Get Booking (Authenticated) | `GET /api/booking/1` | Cookie token | 200 OK | Asserts guest records returned when passing `Cookie: token={{authToken}}`. |
| **API-010** | Get Booking (Unauthenticated) | `GET /api/booking/1` | Missing token | 403 Forbidden | Security check: Access blocked without valid administrative cookie. |
| **API-011** | Submit Contact Message | `POST /api/message` | None | 200 OK | Asserts feedback submission response: `{"success": true}`. |

EXPORTED POSTMAN ARTIFACTS:

- **Collection File:** `postman/Restful-Booker-Platform.postman_collection.json`
- **Environment File:** `postman/Restful-Booker-Platform.postman_environment.json`
- **Newman Execution Command:** `npx newman run postman/Restful-Booker-Platform.postman_collection.json -e postman/Restful-Booker-Platform.postman_environment.json`

## 04 Performance Testing Approach & Analysis (Task 4)

Target: `POST /api/booking` | Normal Traffic: 5 RPS | Peak Traffic: 20 RPS | Tool: k6 (Grafana).

**WORKLOAD MODEL (K6 RAMPING ARRIVAL RATE):**

1. **Stage 1 (Normal Traffic):** 5 RPS sustained for 1 minute.
2. **Stage 2 (Traffic Ramp-up):** Linear increase from 5 RPS to 20 RPS over 2 minutes.
3. **Stage 3 (Peak Traffic Stress):** 20 RPS sustained for 2 minutes.
4. **Stage 4 (Cool-down):** Ramp down from 20 RPS to 0 RPS in 30 seconds.

**QUALITY GATES & THRESHOLDS:**

| METRIC | THRESHOLD | RATIONALE |
|---|---|---|
| `http_req_duration` | p95 < 500ms | Guarantees 95% of guests fast checkout. |
| `http_req_duration` | p99 < 1200ms | Tail latency ceiling under peak DB writes. |
| `http_req_failed` | rate < 1% | Max 1% network/system dropouts allowed. |
| `checks` | rate > 99% | Validates contract & response integrity. |

**EXAMPLE K6 SCRIPT SNIPPET ( `PERFORMANCE/BOOKING-PERF.JS` ):**

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  scenarios: {
    booking_traffic_ramp: {
      executor: 'ramping-arrival-rate',
      startRate: 5,
      timeUnit: '1s',
      preAllocatedVUs: 50,
      maxVUs: 150,
      stages: [
        { target: 5, duration: '1m' },   // Normal Traffic (5 RPS)
        { target: 20, duration: '2m' },  // Ramp-up to Peak (20 RPS)
        { target: 20, duration: '2m' },  // Peak Stress (20 RPS)
        { target: 0, duration: '30s' },  // Recovery
      ],
    },
  },
  thresholds: {
    'http_req_duration': ['p(95)<500', 'p(99)<1200'],
    'http_req_failed': ['rate<0.01'],
    'checks': ['rate>0.99'],
  },
};

export default function () {
  const url = 'https://automationintesting.online/api/booking';
  const payload = JSON.stringify({
    roomid: 1,
    firstname: 'PerfGuest',
    lastname: 'k6Test',
    depositpaid: false,
    bookingdates: { checkin: '2028-06-10', checkout: '2028-06-12' },
    email: `perf_${__VU}_${__ITER}@loadtest.io`,
    phone: '0812345678901',
  });
  const res = http.post(url, payload, { headers: { 'Content-Type': 'application/json' } });
  check(res, { 'status is 201 or 409': (r) => r.status === 201 || r.status === 409 });
  sleep(0.5);
}
```

## 05 Automation Framework Overview (Task 3)

Architecture: **Playwright BDD + TypeScript + Page Object Model (OOP)**.
Features separated Page Objects ( `BasePage` , `HomePage` , `ReservationPage` ), custom fixture injection, and multi-reporters.

**EXECUTION RESULTS:**

**✓ 2 Scenarios Executed & Passed 100% (Green)**

- `[XRAY-101]` Successfully book a room with valid guest details (4.1s)
- `[XRAY-102]` Display validation errors on missing mandatory fields (5.6s)

Output JUnit XML automatically generated at `results/results.xml` for Jira Xray import.

**FRAMEWORK ARCHITECTURE DECISIONS:**

- **OOP Encapsulation:** Step definitions are zero-locator controllers.
- **Dynamic Fixtures:** Auto-computes non-colliding future stay dates.
- **Xray Native Tagging:** 1:1 Jira issue key traceability via JUnit report.

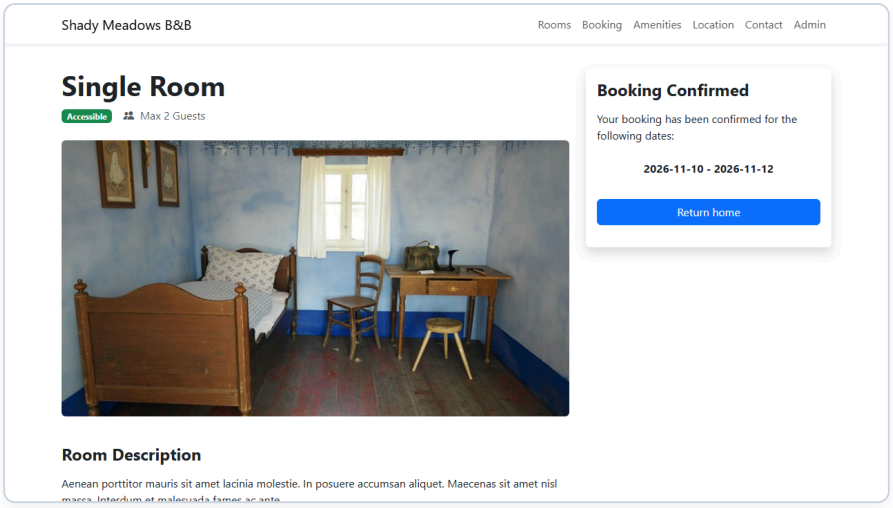



*Figure 5.1: Live execution proof of successful E2E booking reservation with confirmed dates card.*